

# GENIE LOGICIEL

## CHAPITRE N°1 : INTRODUCTION AU GL

Dr. K. Benali

[Benali.khaled@univ-bechar.dz](mailto:Benali.khaled@univ-bechar.dz)

Niveau: 3ING (IA-CS)

Université Tahri Mohammed de Béchar

2025-2026

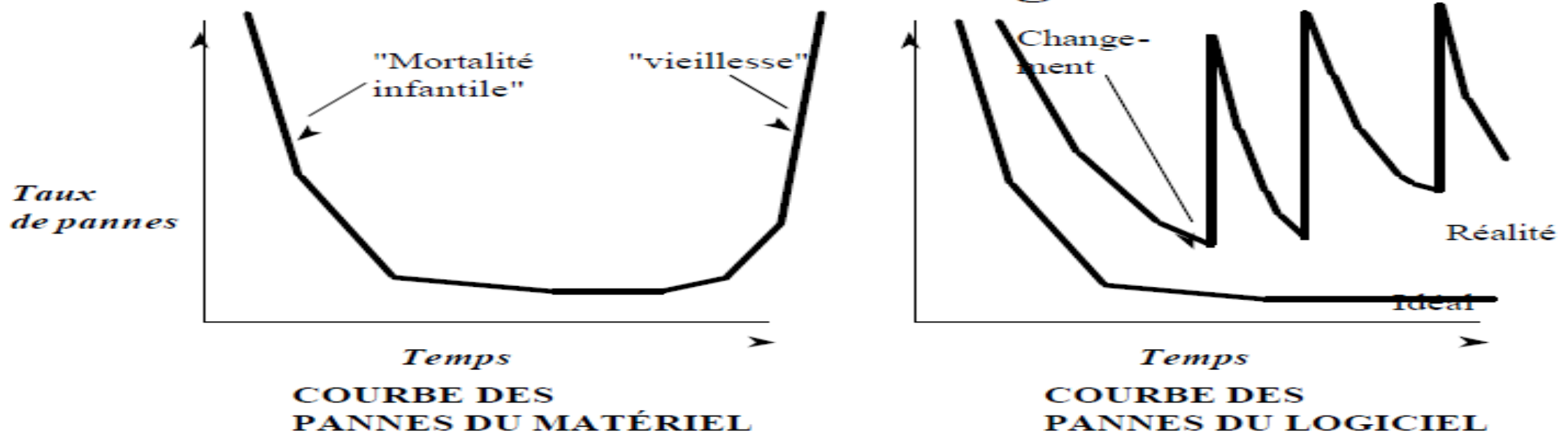
# Utilité de module

# Utilité de module

Systèmes informatiques: 80 % de logiciel, 20 % de matériel

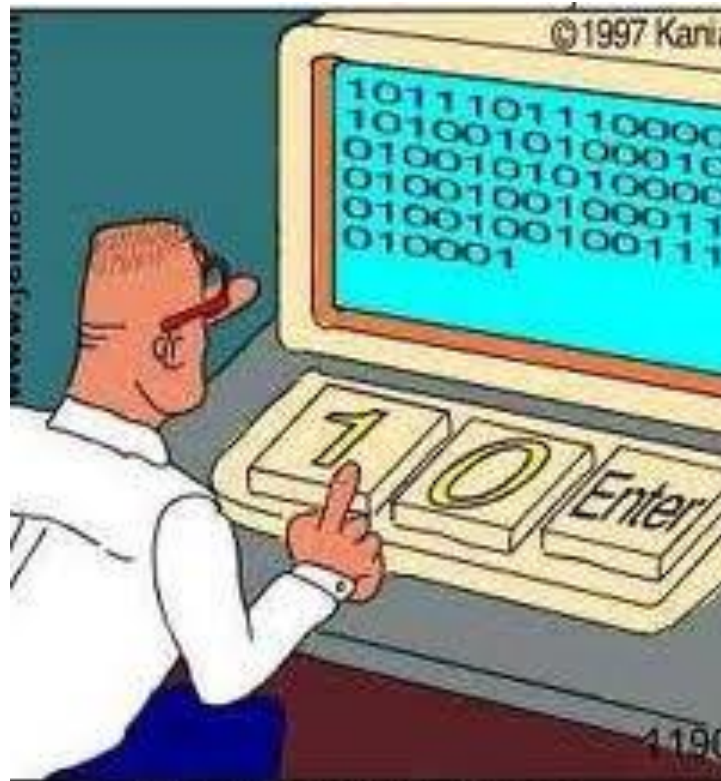
Où se trouve le logiciel?

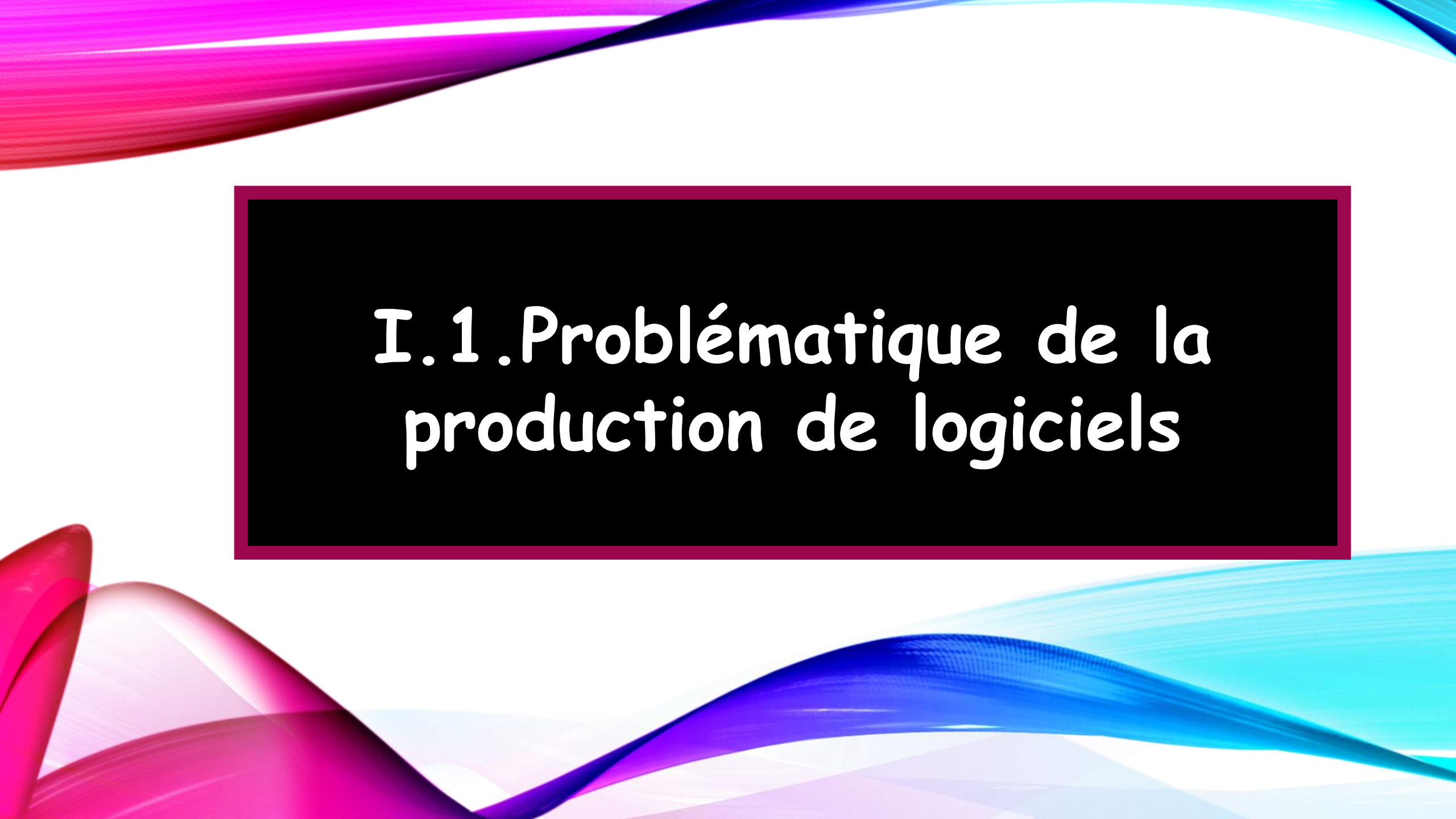
## Fréquence des pannes du matériel et du logiciel



# Programmation VS GE LOG

## Activité personnelle vs Activité d'équipe





## I.1.Problématique de la production de logiciels

# I.1. Problématique de la production de logiciels

6

## BUGS INFORMATIQUES



## I.1. Problématique de la production de logiciels

- Bugs informatiques

- Un bug informatique fait 9 morts et 14 blessés (14 octobre 2007) armée sud africaine : un canon anti aérien s'est retourné, tout en tirant, au hasard.

- Bug de l'an 2000



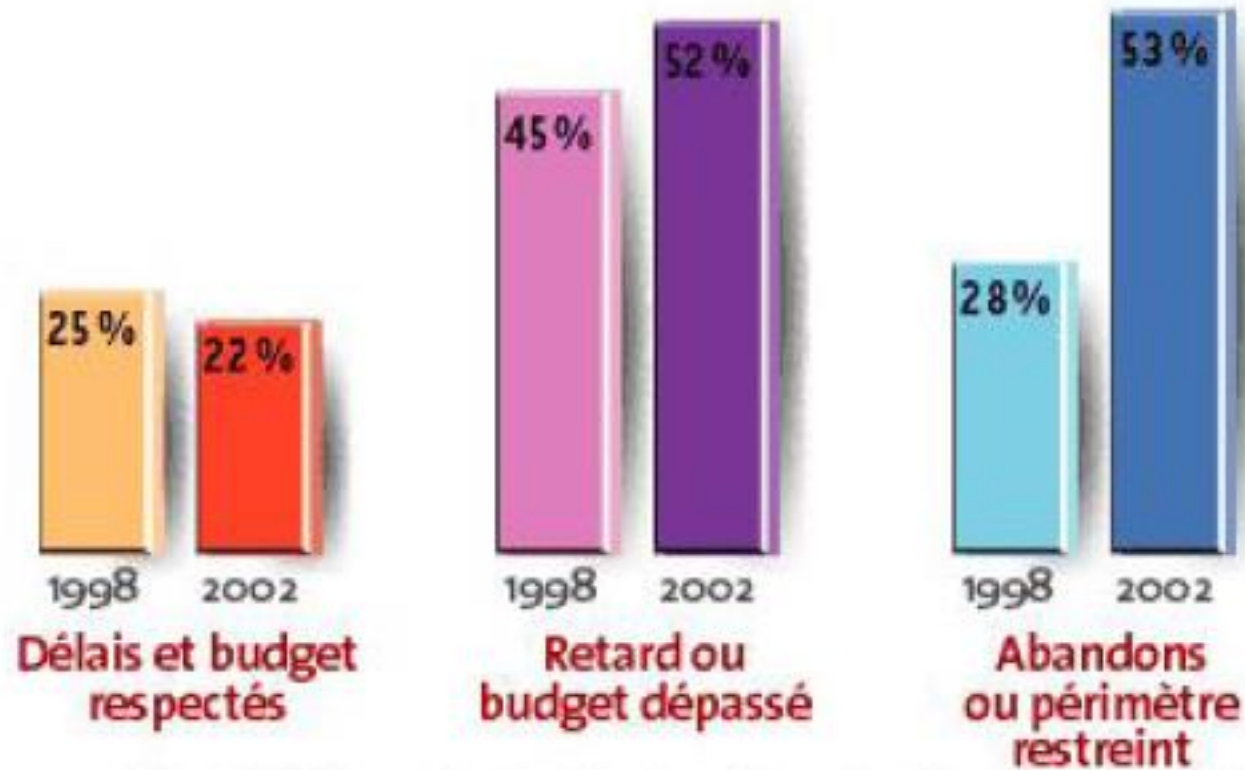
## I.1. Problématique de la production de logiciels

- **Logiciels volumineux (Taille importante des logiciels)**

|                  |                    |
|------------------|--------------------|
| téléphone mobile | 150 K instructions |
| automobile       | 1 M instructions   |
| noyau Linux      | 3,7 M instructions |
| portail Yahoo    | 11 M instructions  |
| Linux            | 100 M instructions |

## I.1. Problématique de la production de logiciels

- **Mauvaise qualité du logiciel**

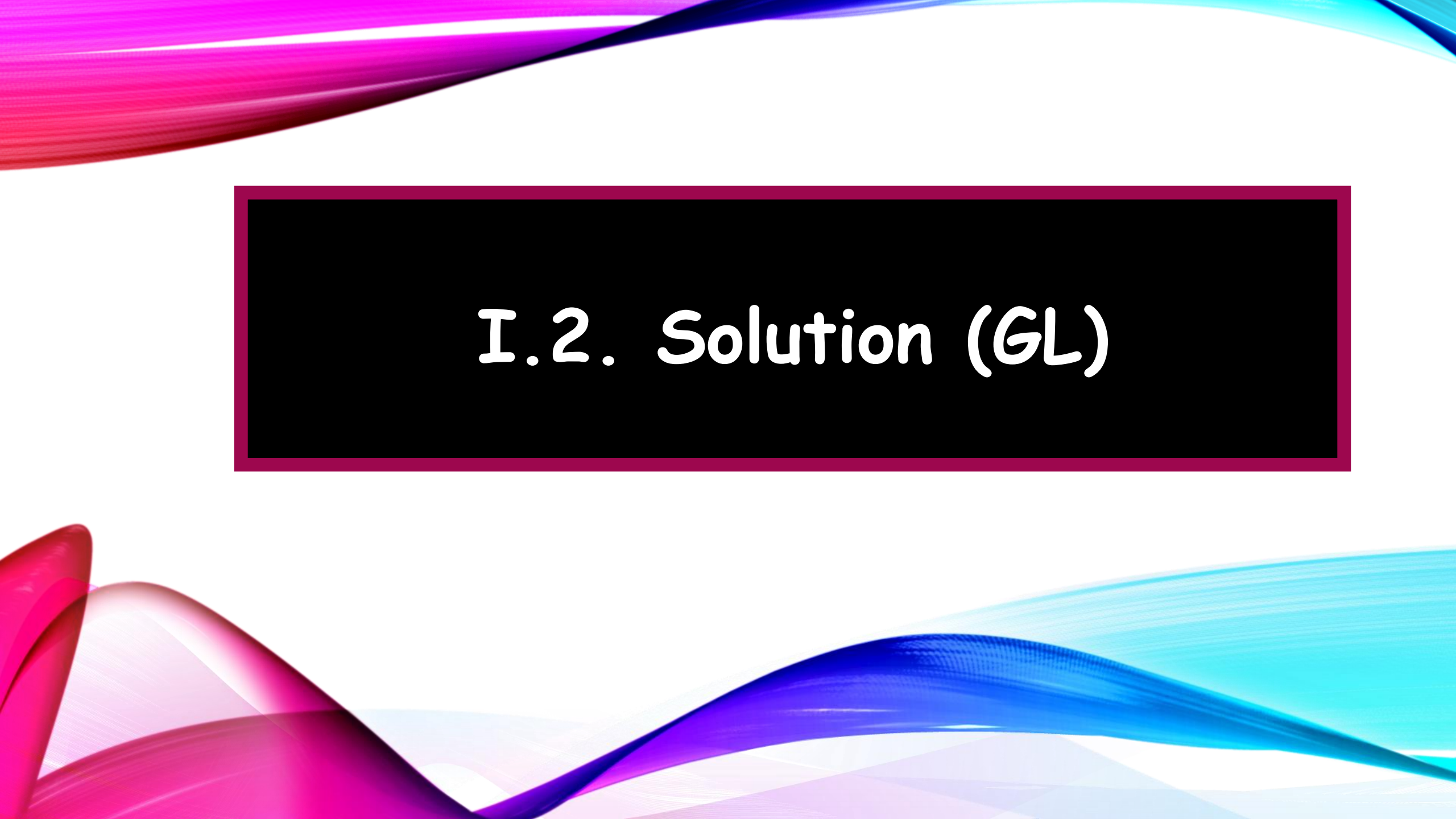


## I.1. Problématique de la production de logiciels

- La crise du logiciel

Coût du logiciel / Coût du matériel

- Le coût du logiciel est supérieur à celui du matériel.
- La maintenance représente près de 70 % du coût relatif du logiciel.
- Un programmeur passe 40 % de son temps de travail à faire de la maintenance.



## I.2. Solution (GL)

## I.2.1. Définition

- **Génie logiciel = Software Engineering  
/= Software Genius**



There is no need to be  
a software genius to do  
software engineering !

## I.2.1. Définition

- **Génie logiciel (Software Engineering)**

- **Génie** fait directement référence à **génie civil** = **l'art** de la construction.
- Construire un bâtiment < > poser briques

$\sum_{\text{activités}}$  (conception architecturale, maçonnerie, plomberie, électricité,...)<sub>coordonnées</sub>

- **Un logiciel** est un **ensemble d'entités** nécessaires (Pg (code source, exe), documentation et informations) au fonctionnement d'un processus de traitement **automatique** de **l'information**.

- un ensemble **des méthodes et des techniques** pour la construction **des systèmes informatiques (des logiciels)** de **bonne qualité**.

## I.2.2. Objectifs de GL



Logiciel de Qlt = Délai respecté + Coût maîtrisé + fiabilité

## I.2.3. Histoire

### Conférence à Garmish, Allemagne (1968)



## I.2.4. Qualités attendues d'un logiciel

**Utilité**

**Utilisabilité**

**Fiabilité**

**Performance**

**Interopérabilité**

**Réutilisabilité**

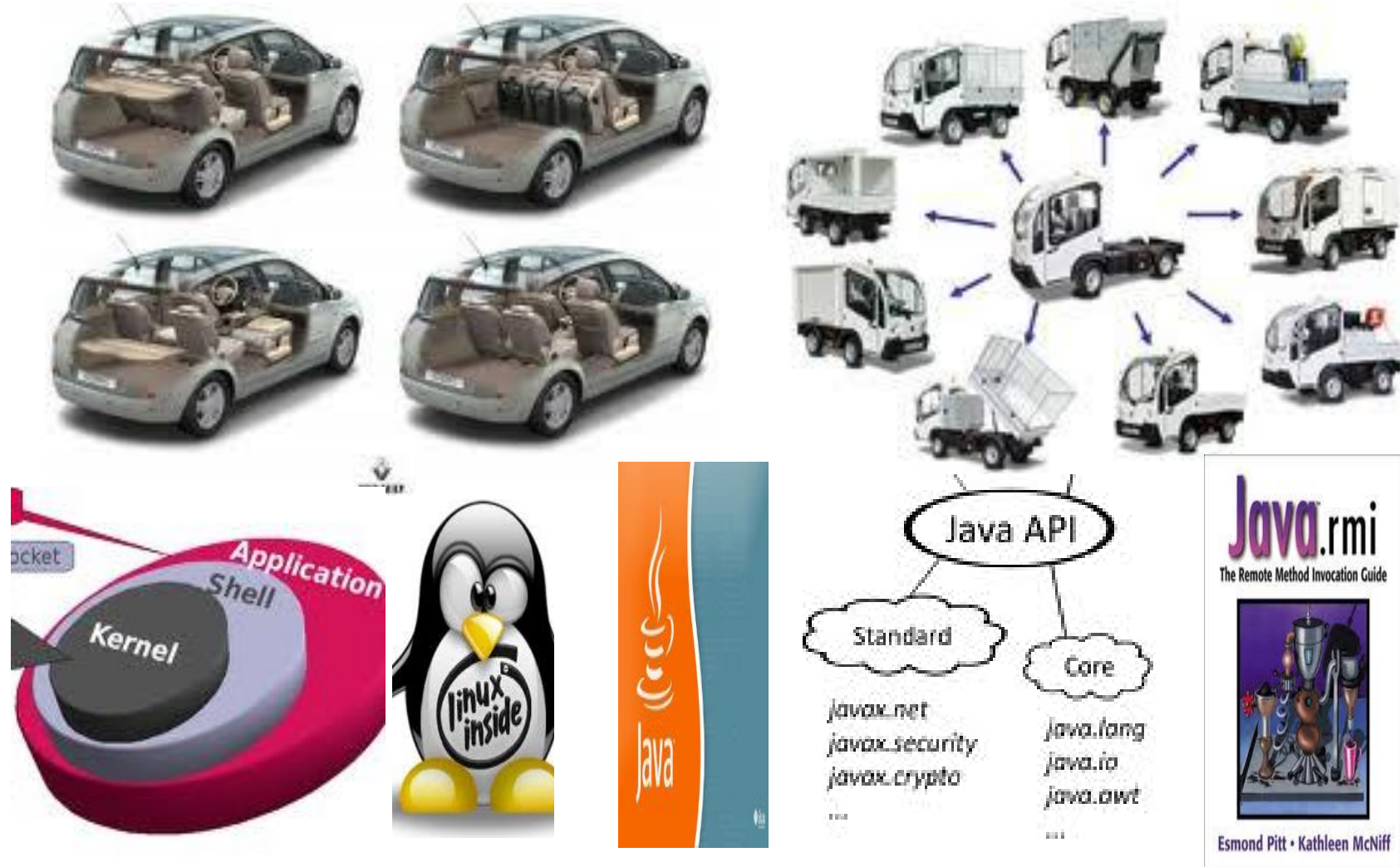
**Maintenance  
facile**

### I.2.5.1 LA GÉNÉRALISATION

### I.2.5.2 STRUCTURATION (LA DÉCOMPOSITION)



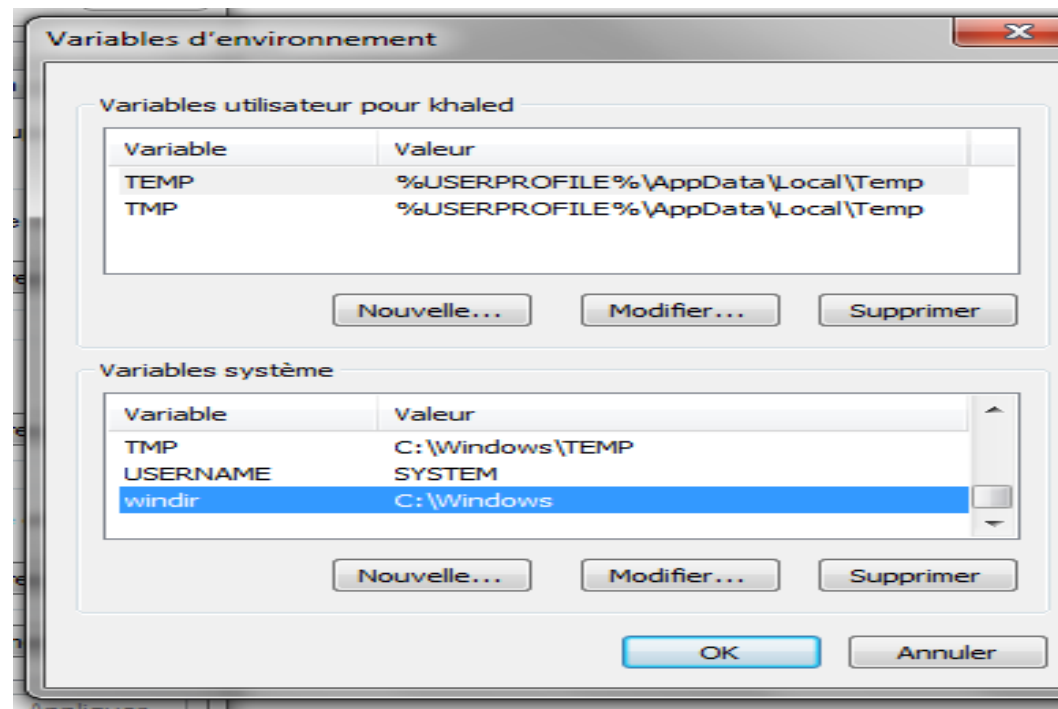
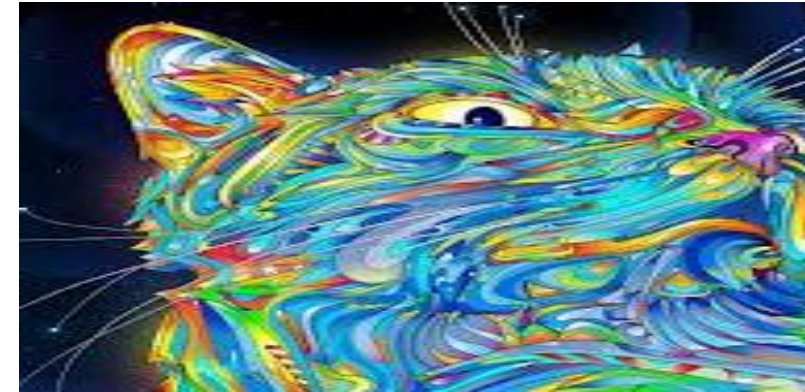
### I.2.5.3. LA MODULARITÉ



## I.2.5. Principes de GL

19

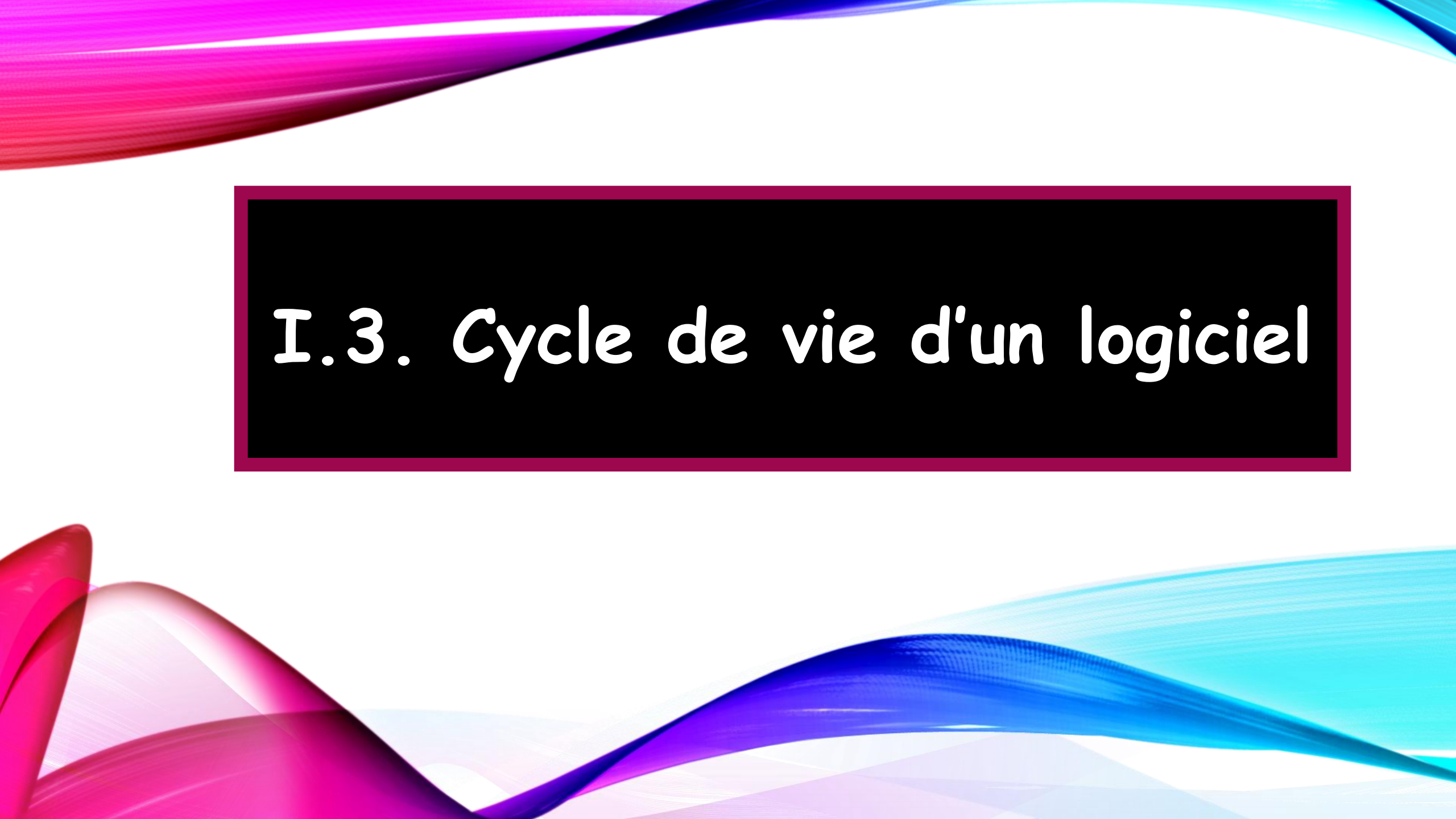
### I.2.5.4 L'ABSTRACTION



## I.2.5. Principes de GL

### I.2.5.5 LA DOCUMENTATION





## I.3. Cycle de vie d'un logiciel

## **I.3.1. Composantes du cycle de vie d'un logiciel**

### **I.3.1. Composantes du cycle de vie d'un logiciel**

**Analyse et  
Spécification des  
besoins**

**Conception**

**Implémentation**

**Tests**

**Livraison**

**Maintenance**

## I.3.1. Composantes du cycle de vie d'un logiciel

### I.3.1.1 ANALYSE ET SPÉCIFICATION DES BESOINS



## I.3.1. Composantes du cycle de vie d'un logiciel

### I.3.1.1 ANALYSE ET SPÉCIFICATION DES BESOINS

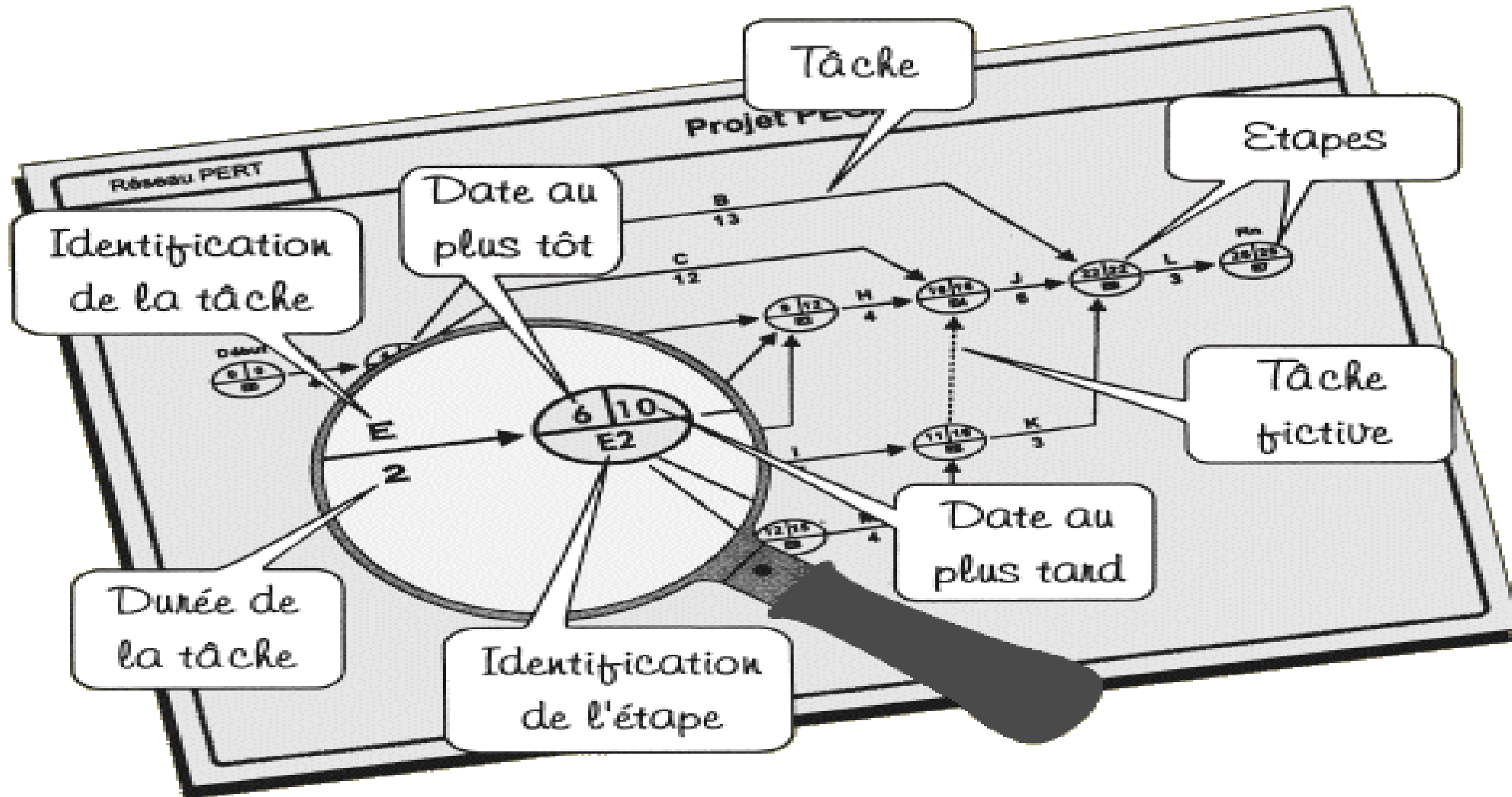
#### La méthode PERT

#### Introduction

#### □ Program Evaluation and Review Technique

1. Identifier les tâches et estimer leur durée
2. Ordonner les tâches
3. Construire le réseau et l'exploiter

# La méthode PERT



# La méthode PERT

## Identification des tâches et de leur durée

| Tâche                                      | Durée | Référence |
|--------------------------------------------|-------|-----------|
| Mettre la farine dans un saladier          | 3 s   | A         |
| Mettre deux oeuf                           | 30 s  | B         |
| Ajouter le lait doucement et mélanger      | 600 s | C         |
| Dans une poêle mettre un petit peu d'huile | 3 s   | D         |
| Couper les bananes en fines lamelles       | 300 s | E         |
| Les mélanger                               | 30 s  | F         |
| Faire chauffer le mélange                  | 120 s | G         |
| Faire flamber                              | 10 s  | H         |
| Faire cuire une crêpe                      | 10 s  | I         |
| Verser du mélange bananes sur la crêpe     | 10 s  | J         |
| Manger                                     | 120   | K         |

# La méthode PERT

## Ordonnancement des tâches

| Pour faire | Il faut faire |
|------------|---------------|
| A          | -             |
| B          | A             |
| C          | B             |
| D          | -             |
| E          | -             |
| F          | D-E           |
| G          | F             |
| H          | G             |
| I          | C             |
| J          | I-H           |
| K          | J             |

## La méthode PERT

❖ les tâches fictives sont des tâches de durée nulle.

## La méthode PERT

| Tâches | Antécédents | Durée |  | Tâches | Antécédents | Durée |
|--------|-------------|-------|--|--------|-------------|-------|
| A      | /           | 3     |  | G      | E-F         | 9     |
| B      | A           | 1     |  | H      | /           | 5     |
| C      | A           | 5     |  | I      | H           | 8     |
| D      | B           | 6     |  | J      | H           | 2     |
| E      | B           | 4     |  | K      | I           | 3     |
| F      | C-I-D       | 2     |  | L      | K-J         | 7     |

# La méthode PERT

## Exploitation d'un réseau PERT

### 1. Calcul des dates au plus tôt de chaque étape

- Elles se calculent de gauche à droite en partant de 0 et rajoutant la durée de la tâche à la date précédente. En cas de convergence, on prend le max

### 2. Calcul des dates au plus tard de chaque étape

- Elles se calculent de droite à gauche en partant de la date de fin au plus tard et en retranchant la durée de la tâche à la date précédente. En cas de convergence, on prend le min

# La méthode PERT

## Exploitation d'un réseau PERT

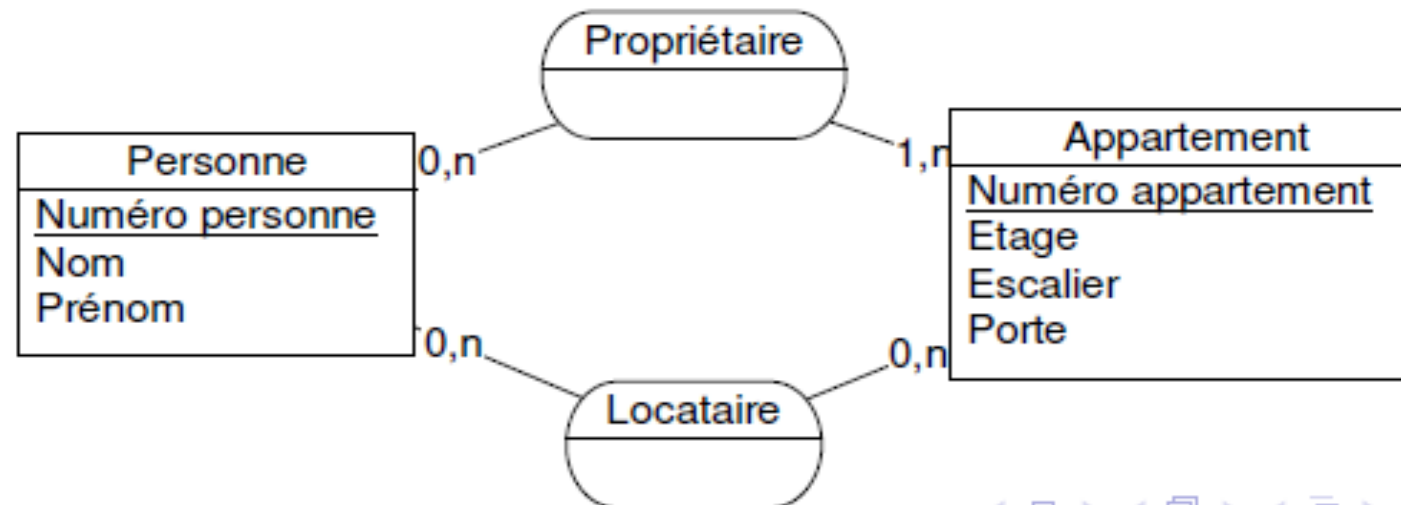
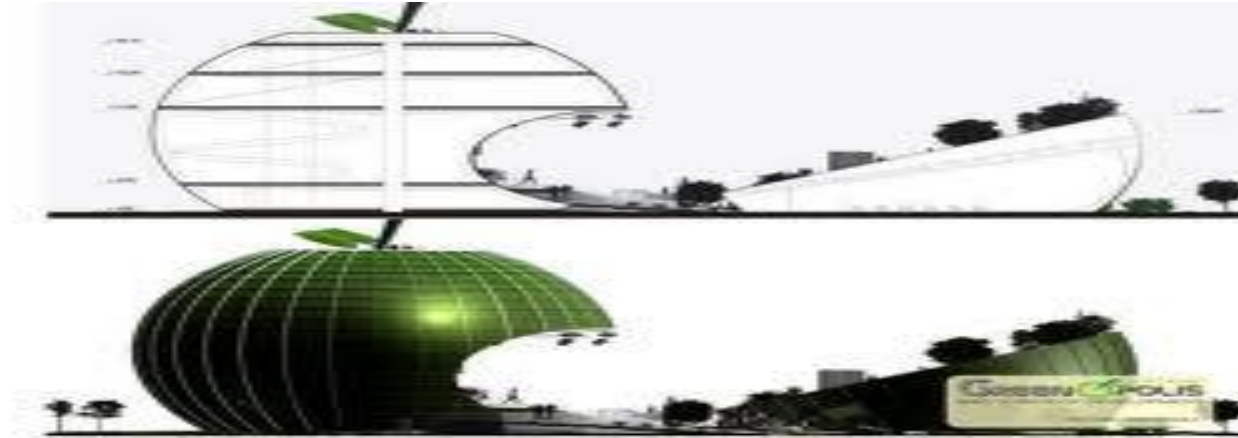
### 3. Calcul des marges et du chemin critique

#### Chemin critique

- relie les étapes qui n'ont aucune marge (marge nulle)
- Un retard dans une tâche sur le chemin critique entraînera un retard dans le projet entier

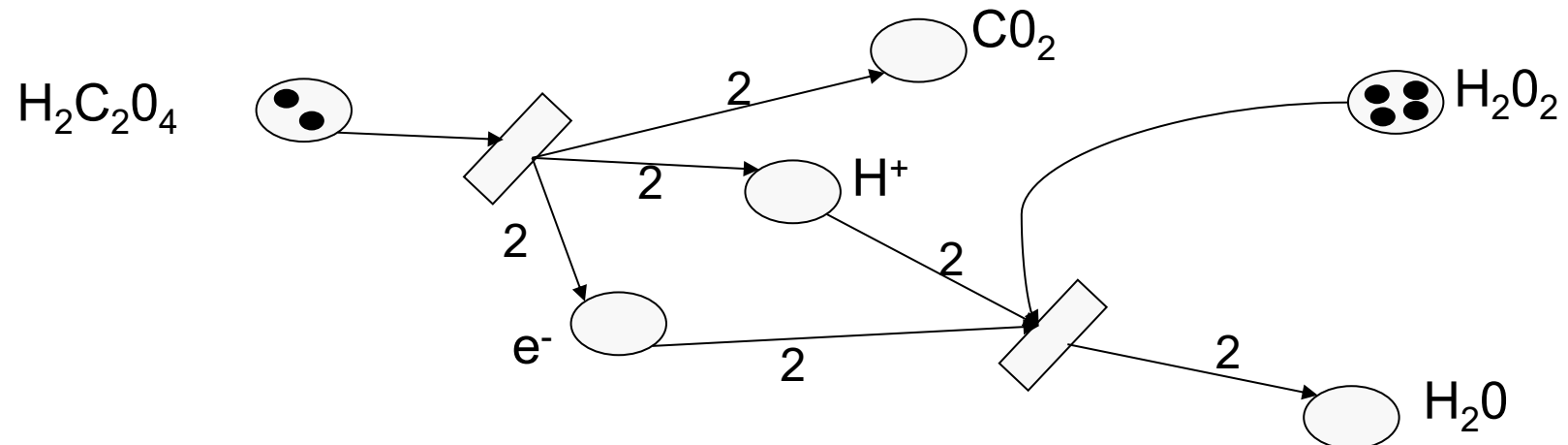
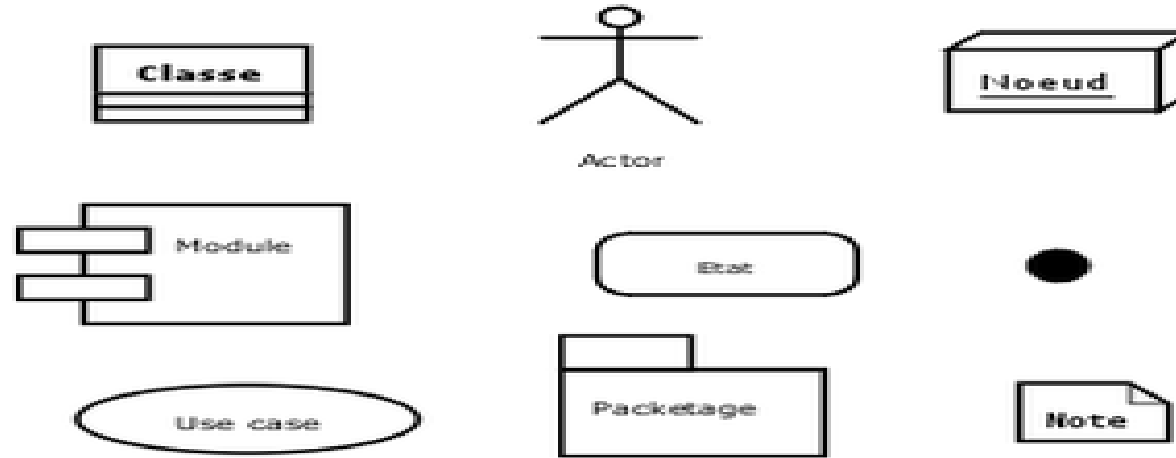
## I.3.1. Composantes du cycle de vie d'un logiciel

### I.3.1.2 CONCEPTION



## I.3.1. Composantes du cycle de vie d'un logiciel

### I.3.1.2 CONCEPTION



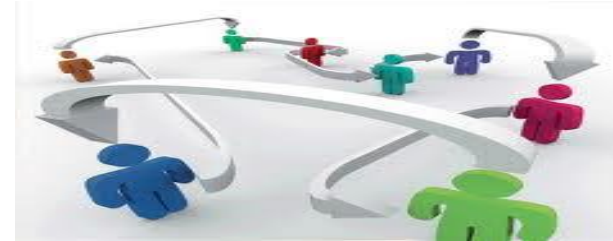
## I.3.1. Composantes du cycle de vie d'un logiciel

### I.3.1.2 CONCEPTION

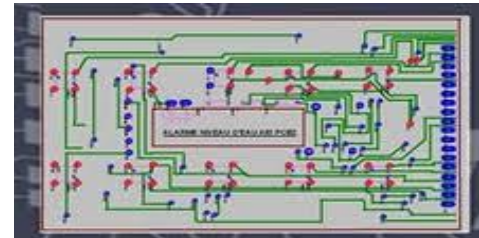
**Conception  
architecturale**



**Conception des  
interfaces**



**Conception détaillée**



## I.3.1. Composantes du cycle de vie d'un logiciel

### I.3.1.3 IMPLÉMENTATION



```

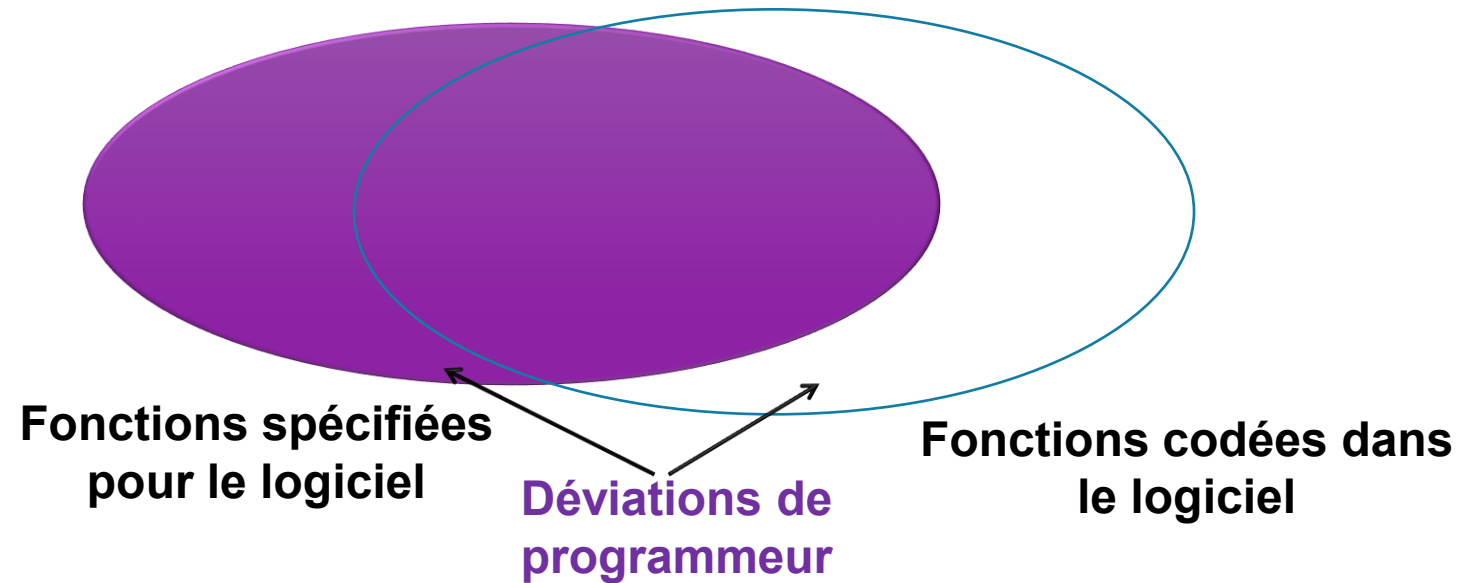
44 /**
45  * Classe Utils
46  *
47  */
48
49 class Utils{
50
51     private $_variables = array();
52
53     public function insertInto($var, $value){
54         if(!in_array($var,$this->_variables)){
55             $this->_variables[$var] = $value;
56         }
57     }
58
59     public function show(){
60         foreach ($this->_variables as $var => $value){
61             echo '$var = '.$var.' ; $value = '.$value;
62         }
63     }
64 }

```

## I.3.1. Composantes du cycle de vie d'un logiciel

### I.3.1.4 TESTS

#### I.3.1.4.1. DÉFINITION



Un **métier** à part ou une **activité créatrice** en imaginant des **scénarios plausibles** pouvant mettre un logiciel **en défaut**

## I.3.1. Composantes du cycle de vie d'un logiciel

### I.3.1.4 TESTS

#### I.3.1.4.2 OBJECTIFS

- ❖ Détecter **des déviations** par rapport aux spécifications
- ❖ Détecter **des erreurs**
- ❖ Evaluer **les performances** (fiabilité, rapidité,... )

## I.3.1. Composantes du cycle de vie d'un logiciel

39

### I.3.1.4. TEST EXEMPLE

- ❖ Ecrire une fonction en **Pascal** qui retourne **la racine carrée** d'un nombre **positif** et le **nombre** lorsqu'il est **négatif**.
- ❖ Faites **un jeu de test**.

## **I.3.1. Composantes du cycle de vie d'un logiciel**

### **I.3.1.4 TESTS**

#### **I.3.1.4.3 CLASSIFICATION SELON L'ORDRE**

**Tests Unitaires**

**Tests d'Intégration**

**Tests Système**

**Tests Alpha**

**Tests Bêta**

**Tests d'Acceptation**

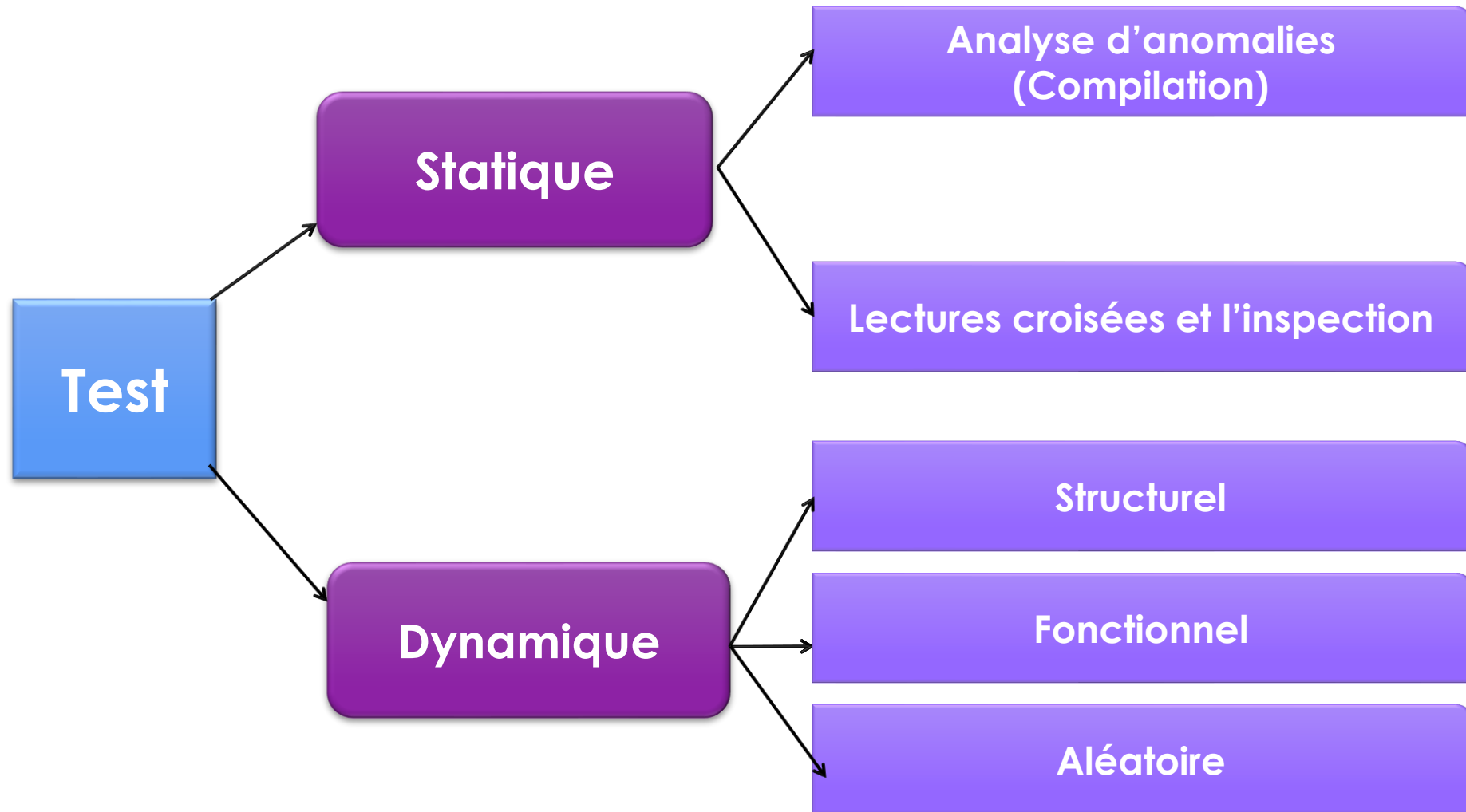
**Test de Régression**

## I.3.1. Composantes du cycle de vie d'un logiciel

41

### I.3.1.4. TEST

#### I.3.1.4.4 CLASSIFICATION SELON LE CONTENU



## I.3.1. Composantes du cycle de vie d'un logiciel

42

### I.3.1.4. TEST

#### I.3.1.4.4 CLASSIFICATION SELON LE CONTENU

##### A. TEST STATIQUE

**program** recherche;

**const**

max\_elt = 50;

**type**

Tliste = **array**[1..max\_elt] **of**

**integer**;

**var**

liste : Tliste;

✓ **But** du programme

non exprimé

✓ **Manque de**

**commentaires**

✓ **Identificateurs**

non explicites

### I.3.1.4. TEST

#### I.3.1.4.4 CLASSIFICATION SELON LE CONTENU

##### B. TEST DYNAMIQUE

Fonctionnel

Structurel

Aléatoire

Figure: Types d'un Test Dynamique

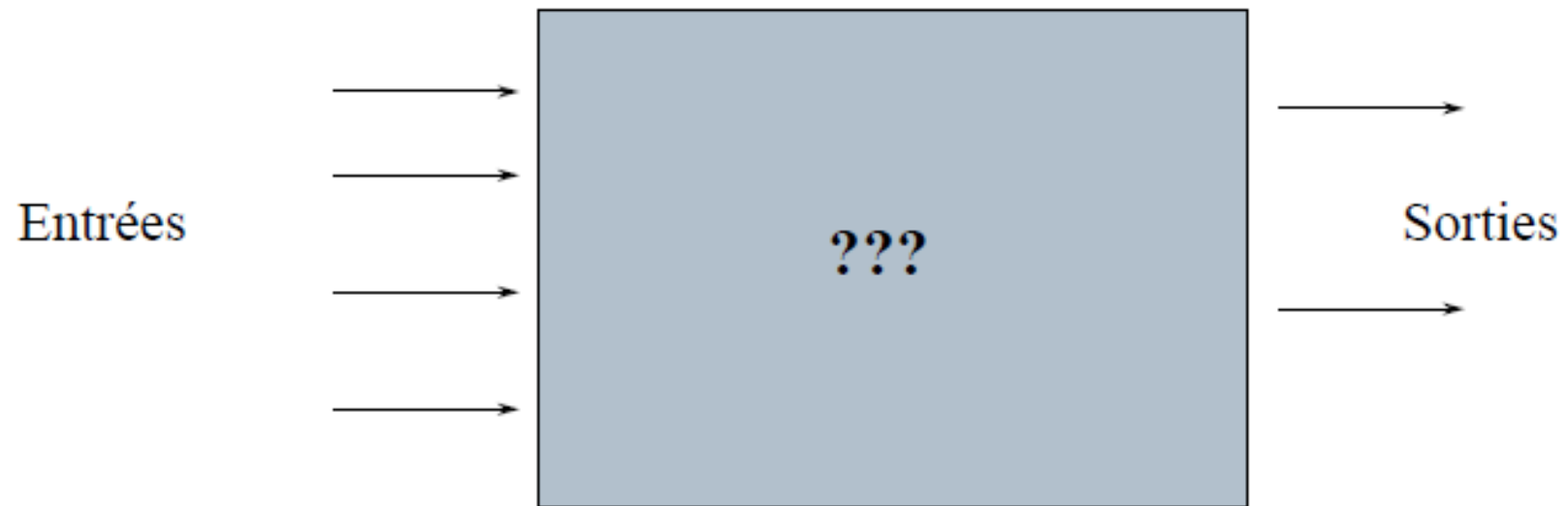
## I.3.1. Composantes du cycle de vie d'un logiciel

44

### I.3.1.4. TEST

#### I.3.1.4.4 CLASSIFICATION SELON LE CONTENU

##### B. 1. TEST DYNAMIQUE FONCTIONNEL



*Ce que le programme devrait faire...*

On se base sur **les spécifications**, pas sur le pg :  
le pg est une **boite noire** !

### I.3.1.4. TEST

#### I.3.1.4.4 CLASSIFICATION SELON LE CONTENU

##### B. 1. TEST DYNAMIQUE FONCTIONNEL

**Exemple:**

**La spécification:**

Lire 3 entiers (les longueurs des côtés d'un triangle) et tester si le triangle est **quelconque**, **isocèle**, **équilatéral** ou **impossible**.

Aucune des longueurs ne doit être **nulle**.

Faite un **test fonctionnel** pour ce pg.

## I.3.1. Composantes du cycle de vie d'un logiciel

46

### I.3.1.4. TEST

#### I.3.1.4.4 CLASSIFICATION SELON LE CONTENU

##### B. 1. TEST DYNAMIQUE FONCTIONNEL

| Jeu de test       | Sortie                                      |
|-------------------|---------------------------------------------|
| (3,4,5)           | Triangle <b>quelconque</b>                  |
| (5,5,5)           | <b>Equilatéral</b>                          |
| (6,6,7) / (6,7,6) | <b>Isocèle</b>                              |
| (1,2,4) / (1,2,3) | <b>Impossible (<math>x+y \leq z</math>)</b> |
| (0,5,4)           | <b>Erreur</b> : Longueur <b>nulle</b>       |

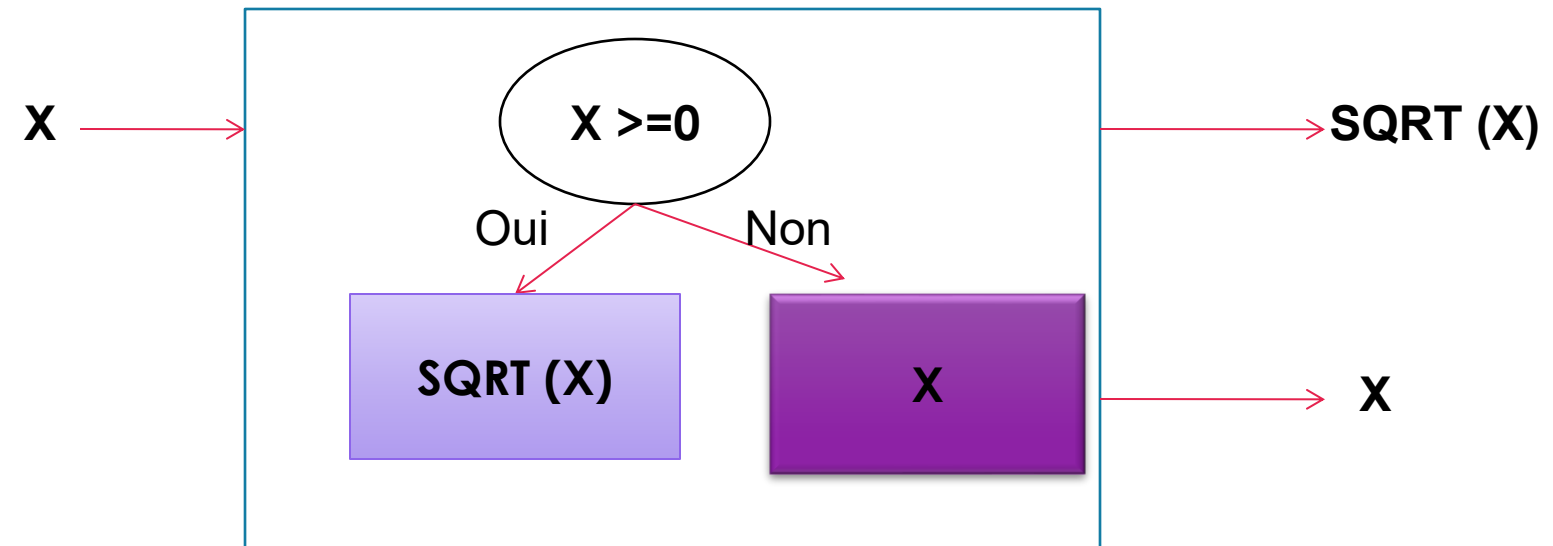
## I.3.1. Composantes du cycle de vie d'un logiciel

47

### I.3.1.4. TEST

#### I.3.1.4.4 CLASSIFICATION SELON LE CONTENU

#### B. 2. TEST DYNAMIQUE STRUCTUREL



On se base sur **le pg (quoi et comment)** :  
le pg est une **boite transparente** !

### I.3.1.4. TEST

#### I.3.1.4.4 CLASSIFICATION SELON LE CONTENU

#### B. 2. TEST DYNAMIQUE STRUCTUREL

##### Graphe de contrôle:

est un graphe **orienté** pour représenter tous **les chemins d'exécution potentiels**.

Ce graphe est composé de:

**Sommets:** **blocs** élémentaires du Pg (Séquence max d'instructions séquentielles)

**Arcs:** **enchaînement d'exécution** possible entre deux sommets.

Rq: Une seule **E** et une seule **S**

## I.3.1. Composantes du cycle de vie d'un logiciel

49

### B. 2. TEST DYNAMIQUE STRUCTUREL

Exemple d'un Graphe de contrôle:

```
open (fichier1) ;  
read (x,fichier1) ;  
read (y,fichier1) ;  
z := 0 ;
```

```
-----  
while x ≥ y do
```

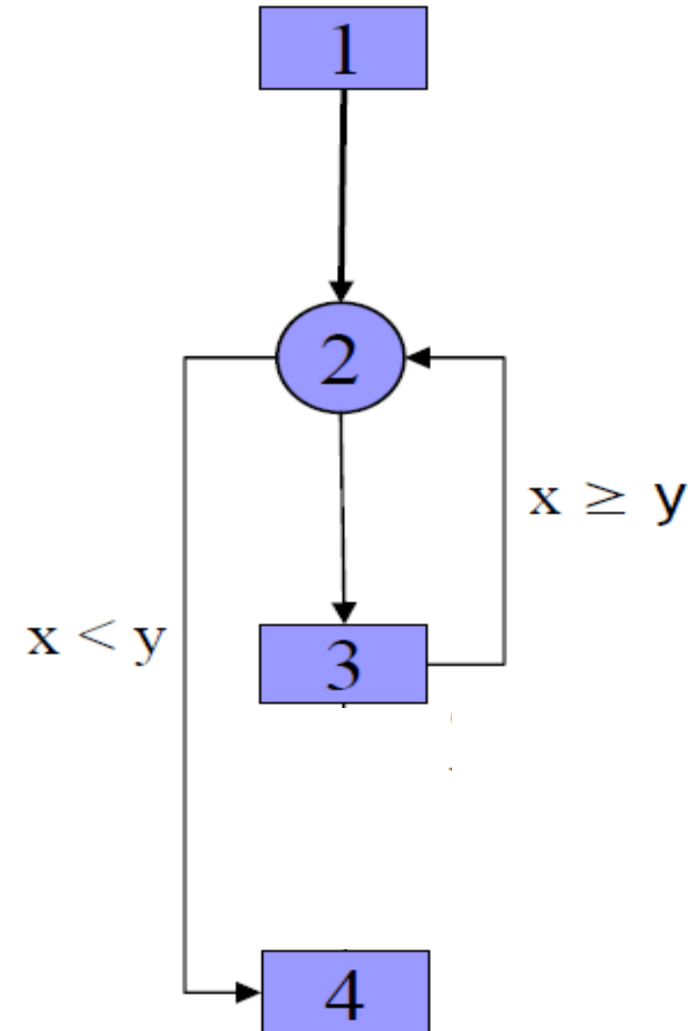
```
-----  
    begin
```

```
        x := x-y ;
```

```
        z := z+1 ;
```

```
    end
```

```
-----  
open (fichier2) ;  
print (y,fichier2) ;  
close (fichier1) ;  
close (fichier2) ;
```



### I.3.1.4. TEST

#### I.3.1.4.4 CLASSIFICATION SELON LE CONTENU

#### B. 2. TEST DYNAMIQUE STRUCTUREL

#### Couverture d'un graphe de contrôle:

Ce test est basé sur différents **critères de couvertures:**

- ❖ Toutes les instructions
- ❖ Tous les enchaînements (arcs)
- ❖ Tous les chemins (i-chemins)

## I.3.1. Composantes du cycle de vie d'un logiciel

51

### I.3.1.4. TEST

#### I.3.1.4.4 CLASSIFICATION SELON LE CONTENU

#### B. 3. TEST DYNAMIQUE ALEATOIRE (STATISTIQUE)



On utilise **des fonctions statistiques** pour sélectionner un jeu de tests.

#### Avantages:

- ❖ Rapide
- ❖ objectivité

## I.3.1. Composantes du cycle de vie d'un logiciel

### I.3.1.5 LIVRAISON



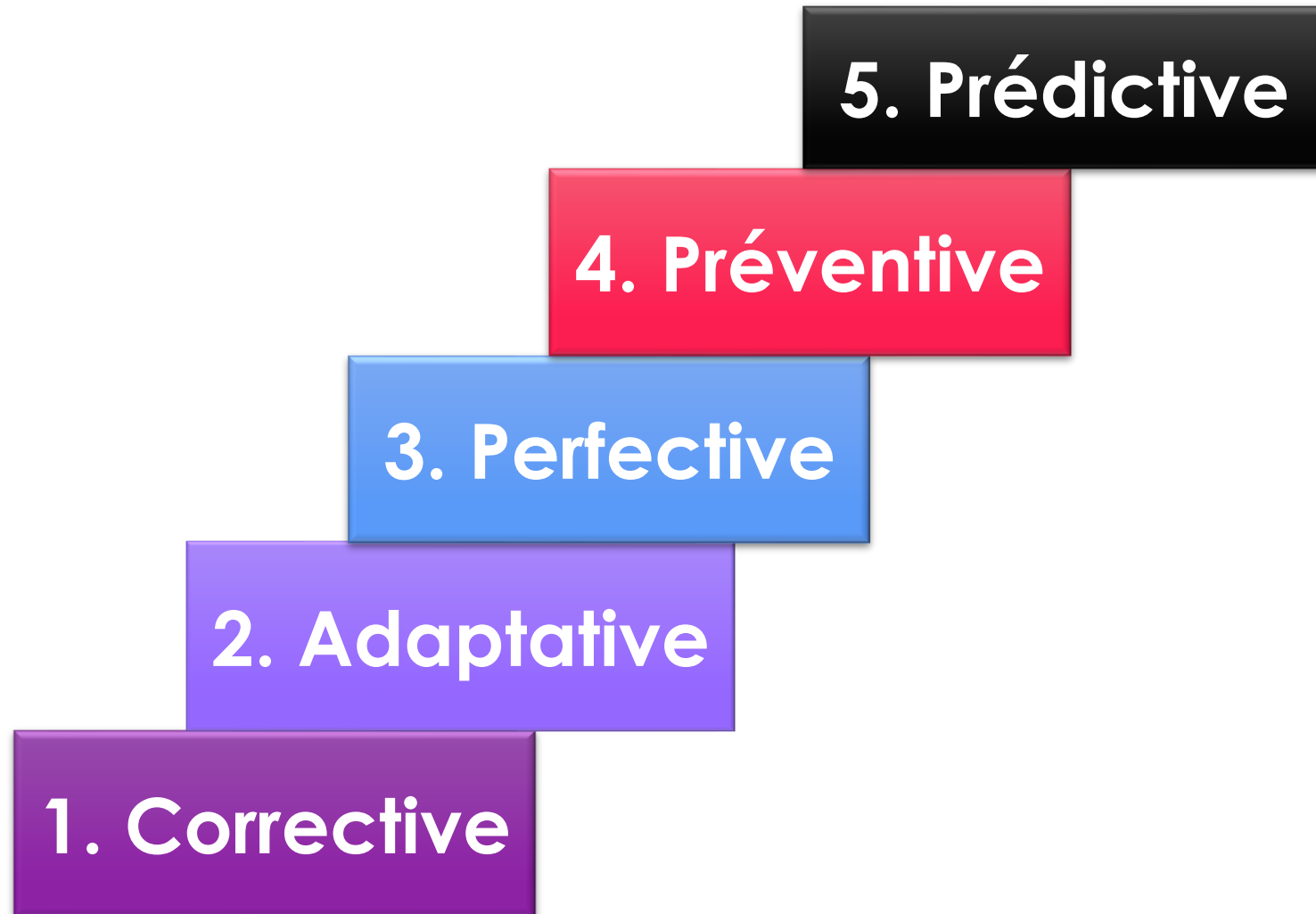
## I.3.1. Composantes du cycle de vie d'un logiciel

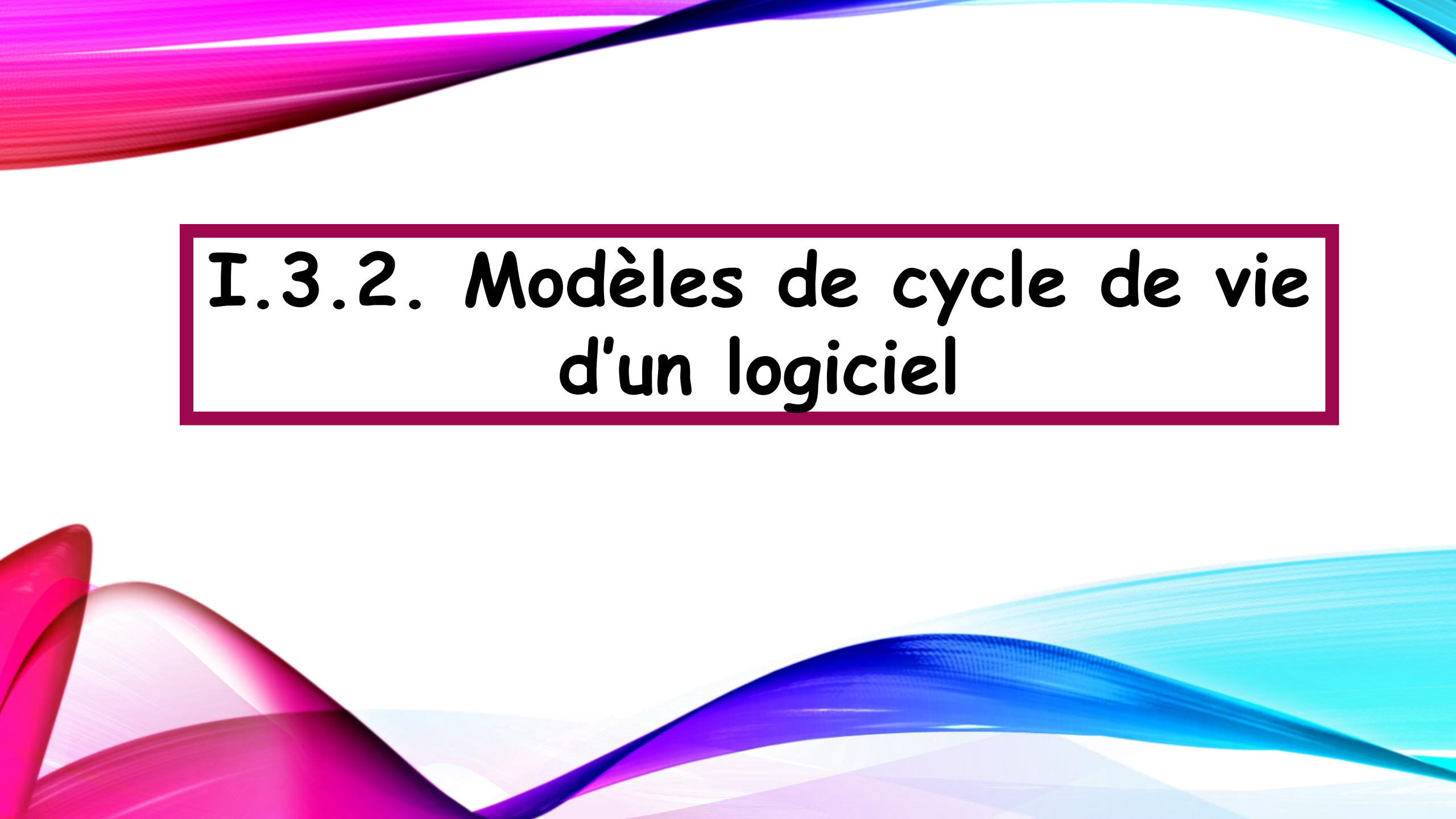
### I.3.1.6 MAINTENANCE



## I.3.1. Composantes du cycle de vie d'un logiciel

### I.3.1.6 MAINTENANCE





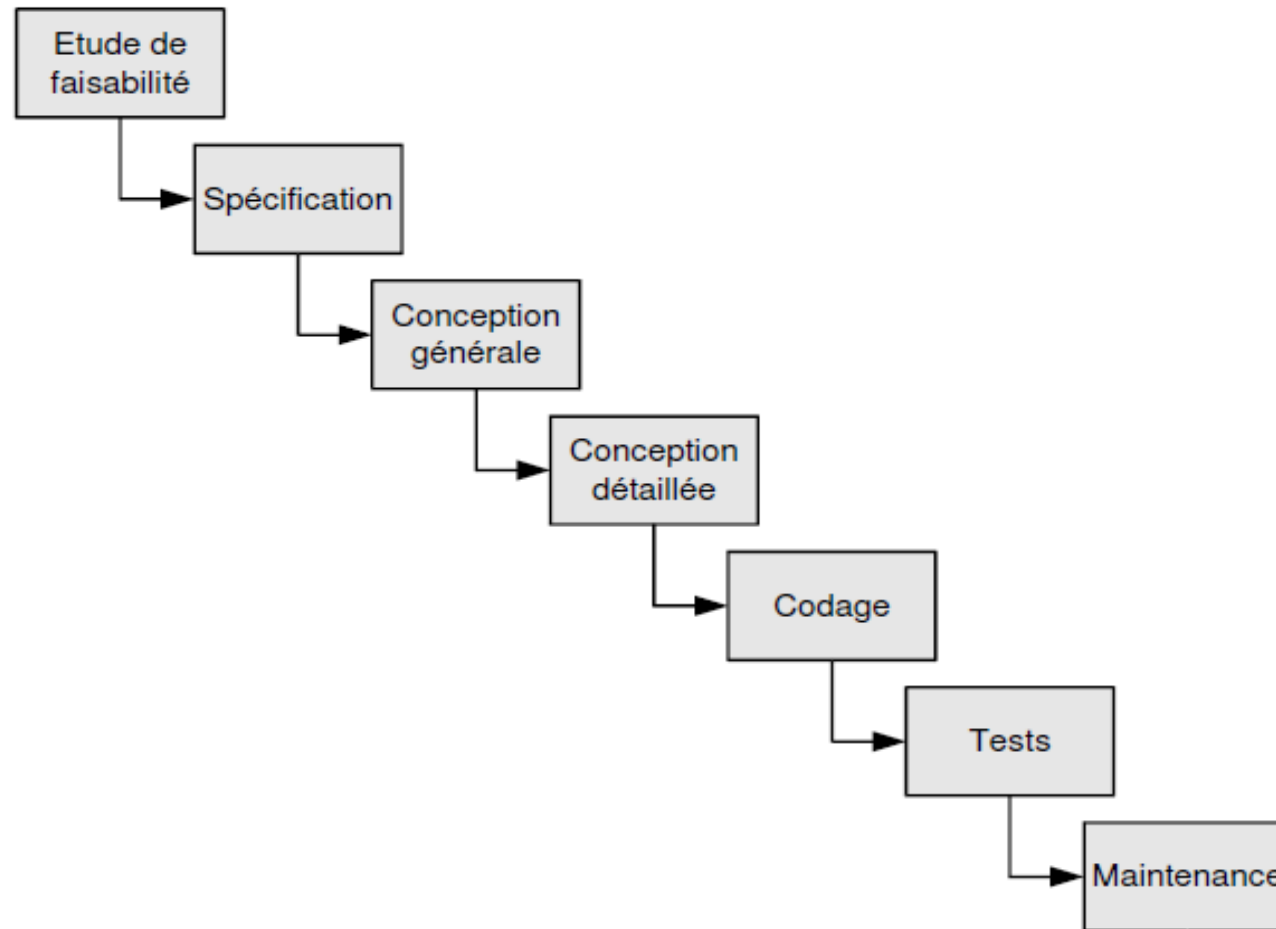
## **I.3.2. Modèles de cycle de vie d'un logiciel**

## I.3.2. Modèles de cycle de vie d'un logiciel

56

### I.3.2.1. MODÈLES LINÉAIRES

#### A. CASCADE [ROYCE, 1970]



## I.3.2. Modèles de cycle de vie d'un logiciel

57

### I.3.2.1. MODÈLES LINÉAIRES

#### A. CASCADE [ROYCE, 1970]

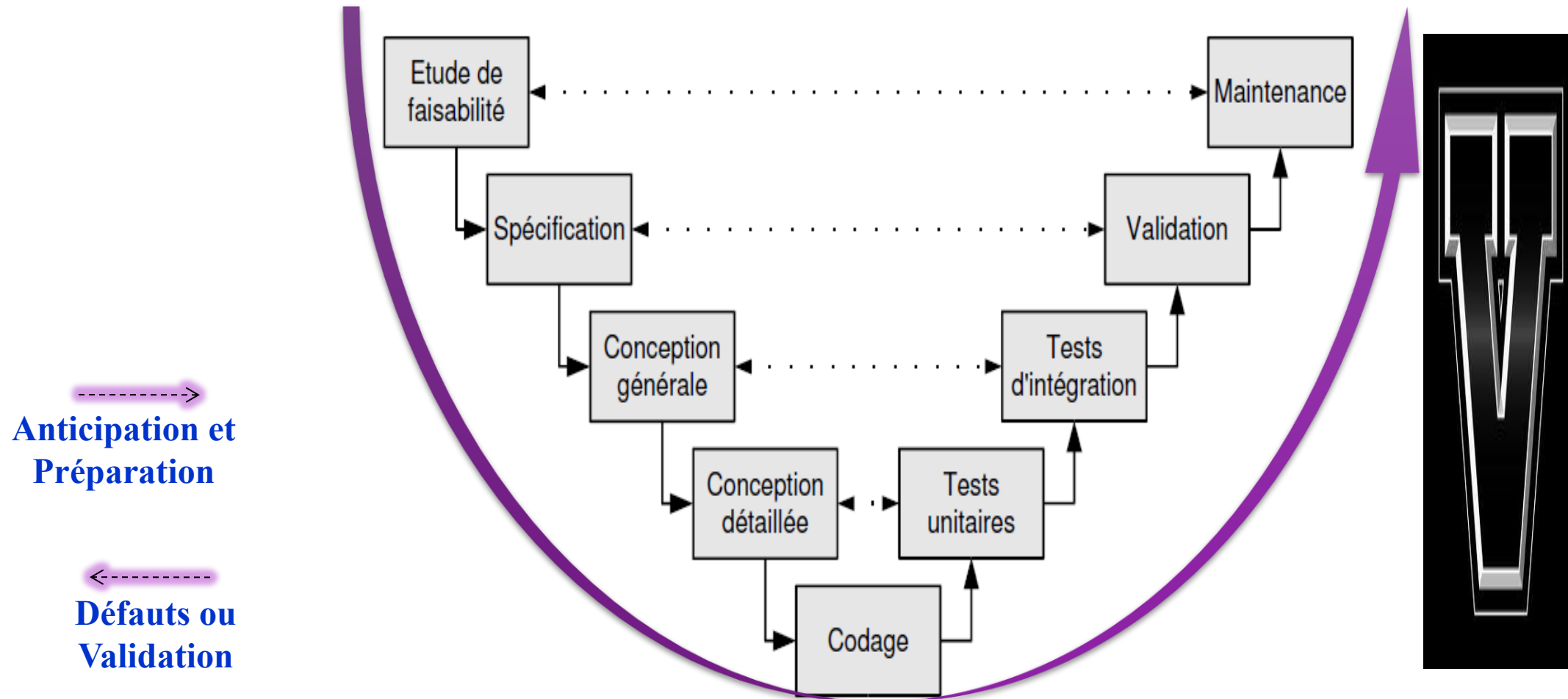
- **Avantages:**
  - ❖ Projet **simple** (petite taille) et bien **maîtrisé**
- **Inconvénients:**
  - ❖ Validation **implicite**
  - ❖ Non **retour**
  - ❖ Pas de **parallélisme** (tâches **séquentielles**)

## I.3.2. Modèles de cycle de vie d'un logiciel

58

### I.3.2.1. MODÈLES LINÉAIRES

#### B. MODÈLE EN V [1980]



## I.3.2. Modèles de cycle de vie d'un logiciel

59

### I.3.2.1. MODÈLES LINÉAIRES

#### B. MODÈLE EN V [1980]

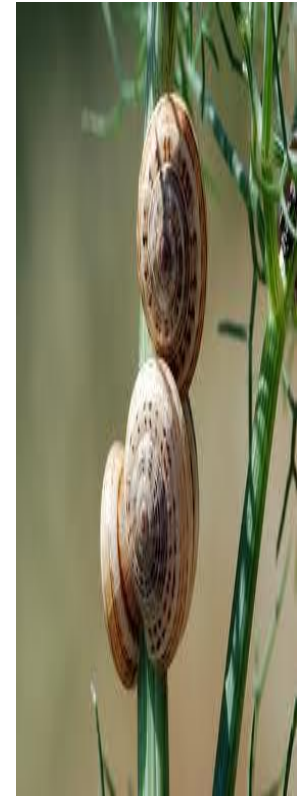
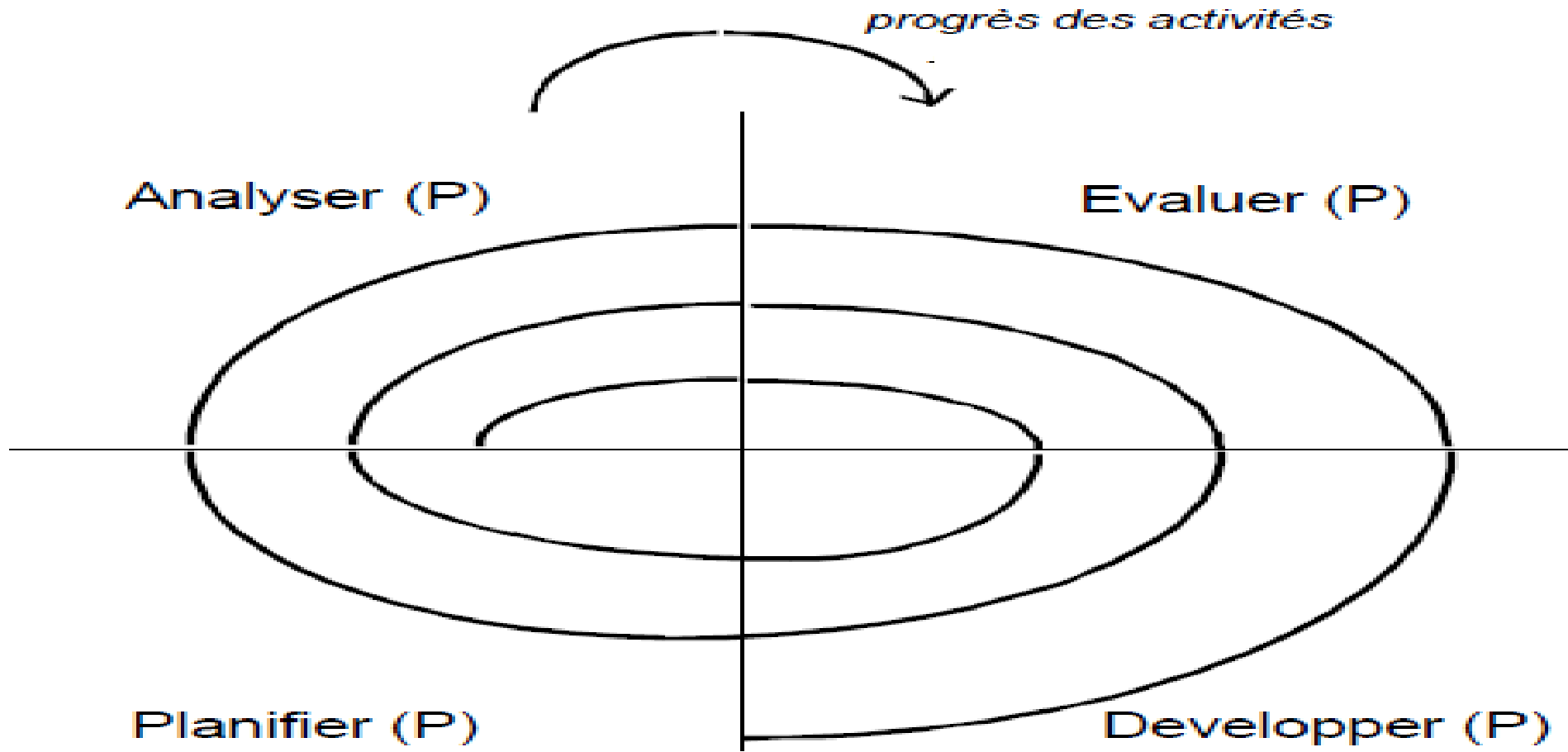
- **Avantages:**
  - ❖ Validation **explicite**
  - ❖ Projets (domaine bien **maîtrisé**)
- **Inconvénients:**
  - ❖ Pas **d'itérations**
  - ❖ Vérification trop **tardive**

## I.3.2. Modèles de cycle de vie d'un logiciel

60

### I.3.2.2. MODÈLES NON LINÉAIRES

#### A. MODÈLE EN SPIRALE [BOEHM, 1988]



## I.3.2. Modèles de cycle de vie d'un logiciel

61

### I.3.2.2. MODÈLES NON LINÉAIRES

#### A. MODÈLE EN SPIRALE [BOEHM, 1988]

$P$  := PROTOTYPE (VERSION D'ESSAI)

while  $P \neq$  produit final

do

Analyser( $P$ );

**objectifs** du cycle à partir des résultats des **cyc-pré**, ou de **l'analyse des besoins**

Evaluer( $P$ );

**Risques** probables (Simulation, maquettage)

Developper( $P$ );

Choisir un **modèle** (cascade ou en V)

Planifier( $P$ )

cycle **suivant**

od;

## I.3.2. Modèles de cycle de vie d'un logiciel

62

### I.3.2.2. MODÈLES NON LINÉAIRES

#### A. MODÈLE EN SPIRALE [BOEHM, 1988]

- **Avantages:**
  - ❖ Itératif
  - ❖ Mixte
- **Inconvénients:**
  - ❖ Compétence requises
  - ❖ Coûteux

## I.3.2. Modèles de cycle de vie d'un logiciel

63

### I.3.2.2. MODÈLES NON LINÉAIRES B. LE MODÈLE INCRÉMENTAL DE PARNAS

1. un logiciel est construit **étape par étape**.
2. Le logiciel est **spécifié et conçu** dans **son ensemble**.
3. **La réalisation** se fait par **incréments** de fonctionnalités.
4. A chaque étape le produit est **testé** et **maintenu** dans son ensemble.
5. Ce modèle permet de faire accepter **progressivement** un logiciel par **les users**.

## 64

# Le modèle en B

